

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 09

NOMBRE COMPLETO: Robles Chávez Uri Shuhua Alexei

Nº de Cuenta: 320195318

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 24 de Octubre de 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

Descripción

En esta parte de la práctica se realizaron dos animaciones complejas utilizando modelos tridimensionales previamente trabajados. El primer ejercicio consistió en hacer que una nave espacial recorriera automáticamente un circuito compuesto por rectas, curvas y una rampa de subida y bajada. Para lograrlo fue necesario calcular tanto la posición como la orientación de la nave durante todo el recorrido.

En el segundo ejercicio se retomó el modelo jerárquico de un satélite formado por un cuerpo y cuatro alas. El movimiento de sus componentes, que anteriormente se realizaba mediante teclado, se automatizó junto con una trayectoria de vuelo, permitiendo que el satélite se desplace y mueva sus alas de manera continua sin intervención del usuario.

Desarrollo

Ejercicio 1. Nave espacial siguiendo un circuito

Para este ejercicio se reutilizó la nave espacial Hippocrates de prácticas anteriores. También se creó un circuito específicamente para que la nave pudiera recorrerlo de forma automática.

El circuito se construyó a partir de una serie de puntos centrales almacenados en puntosCircuito. Se establecieron cuatro tramos principales: dos rectas y dos curvas. Las rectas tienen una longitud aproximada de 80 unidades y las curvas utilizan un radio de 24 unidades.

```
294     const int muestrasPorTramo = 64;
295     const float piCircuito = 180.0f * toRadians;
296     puntosCircuito.clear();
297     for (int tramo=0; tramo<4; ++tramo)
298     {
299         for (int i=0; i<muestrasPorTramo; ++i)
300         {
301             const float u = static_cast<float>(i) / muestrasPorTramo;
302             glm::vec3 centro(0.0f, -1.85f, 0.0f);
303             if (tramo == 0)
304             {
305                 centro.x = -40.0f + 80.0f * u;
306                 centro.z = -119.0f;
307                 // Subida y bajada unidas suavemente, sin escalones en la cima.
308                 const float elevacion = std::sin(piCircuito * u);
309                 centro.y += 8.0f * elevacion * elevacion;
310             }
311             else if (tramo == 1)
312             {
313                 const float angulo = -0.5f * piCircuito + piCircuito * u;
314                 centro.x = 40.0f + 24.0f * std::cos(angulo);
315                 centro.z = -95.0f + 24.0f * std::sin(angulo);
316             }
317             else if (tramo == 2)
318             {
319                 centro.x = 40.0f - 80.0f * u;
320                 centro.z = -71.0f;
321             }
322             else
323             {
324                 const float angulo = 0.5f * piCircuito + piCircuito * u;
325                 centro.x = -40.0f + 24.0f * std::cos(angulo);
326                 centro.z = -95.0f + 24.0f * std::sin(angulo);
327             }
328             puntosCircuito.push_back(centro);
329         }
330     }
```

Figura 1. Generación de los puntos centrales utilizados para construir los diferentes tramos del circuito.

Rectas y curvas del circuito

Las rectas se obtienen modificando principalmente la coordenada X, mientras que las curvas utilizan funciones seno y coseno para generar un cambio progresivo en X y Z.

Por ejemplo, una de las curvas utiliza:

centro.x = 40.0f + 24.0f * std::cos(angulo);

centro.z = -95.0f + 24.0f * std::sin(angulo);

De esta forma se consigue una transición circular entre las dos rectas sin realizar cambios bruscos de posición.

Rampa de subida y bajada

En una de las rectas se agregó una variación sobre el eje Y para formar una rampa.

La elevación se calcula utilizando:

const float elevacion = std::sin(piCircuito * u);

centro.y +=8.0f * elevacion * elevacion;

Con este cálculo la pista inicia a su altura normal, asciende progresivamente hasta alcanzar su punto máximo y posteriormente vuelve a bajar de manera continua.

El uso del seno al cuadrado permite evitar un cambio repentino en la parte superior de la rampa.

Construcción visual de la pista

Después de calcular los puntos centrales, se obtiene una dirección lateral para determinar el ancho del circuito.

A partir de estos puntos se generan las diferentes superficies utilizadas para representar el asfalto, los bordes claros y las paredes laterales de la pista. También se agregó una marca de salida sobre uno de los tramos planos.

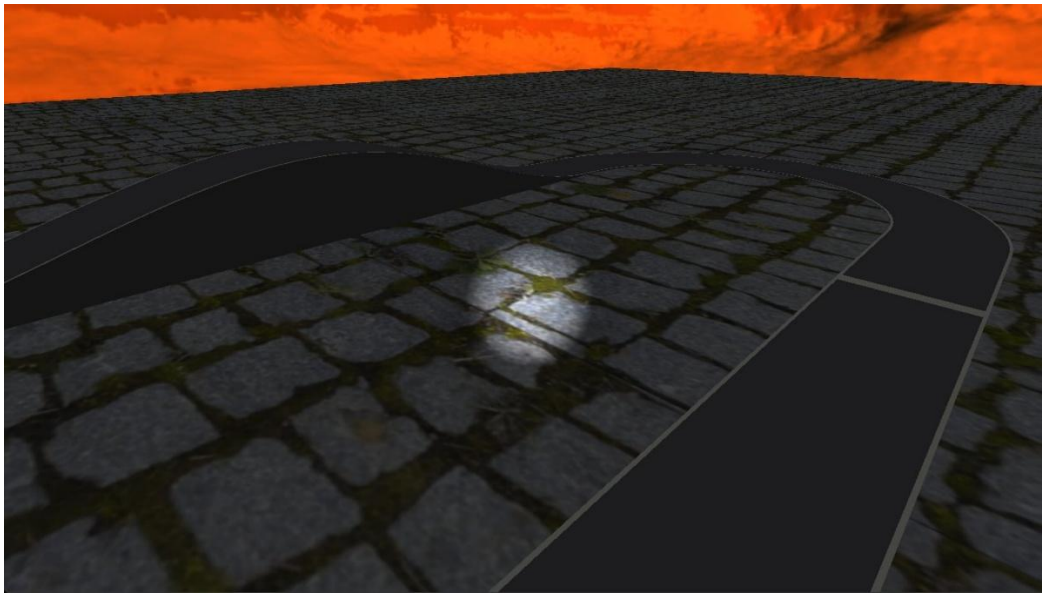


Figura 2. Circuito completo formado por rectas, curvas, bordes y una sección con elevación.

Trayectoria suavizada de la nave

Aunque el circuito ya contiene muchos puntos, para el movimiento de la nave se creó una trayectoria adicional más suave utilizando interpolación Catmull-Rom.

Para cada punto del circuito se toman cuatro posiciones consecutivas:

- p0
- p1
- p2
- p3

y se generan varios puntos intermedios. En el código se utilizan ocho subdivisiones entre cada punto base.

La interpolación permite que la nave pase por el centro del circuito manteniendo transiciones suaves entre las rectas, las curvas y la rampa.

```
634     std::vector<glm::vec3> recorridoNave, direccionesNave;
635     std::vector<double> distanciasNave;
636     const int puntosBaseNave = static_cast<int>(puntosCircuito.size());
637     const int subdivisionesNave = 8;
638     double longitudCircuitoNave = 0.0;
639     for (int i=0; i<puntosBaseNave; ++i)
640     {
641         const glm::vec3 p0 = puntosCircuito[(i+puntosBaseNave-1)%puntosBaseNave];
642         const glm::vec3 p1 = puntosCircuito[i];
643         const glm::vec3 p2 = puntosCircuito[(i+1)%puntosBaseNave];
644         const glm::vec3 p3 = puntosCircuito[(i+2)%puntosBaseNave];
645         for (int k=0; k<subdivisionesNave; ++k)
646         {
647             const float t = static_cast<float>(k)/subdivisionesNave;
648             const glm::vec3 a = -p0 + p2;
649             const glm::vec3 b = 2.0f*p0 - 5.0f*p1 + 4.0f*p2 - p3;
650             const glm::vec3 c = -p0 + 3.0f*p1 - 3.0f*p2 + p3;
651             const glm::vec3 punto = 0.5f*(2.0f*p1 + a*t + b*t*t + c*t*t*t);
652             const glm::vec3 direccion = glm::normalize(0.5f*(a + 2.0f*b*t + 3.0f*c*t*t));
653             if (!recorridoNave.empty()) longitudCircuitoNave += glm::length(punto-recorridoNave.back());
654             recorridoNave.push_back(punto);
655             direccionesNave.push_back(direccion);
656             distanciasNave.push_back(longitudCircuitoNave);
657         }
658     }
```

Figura 3. Generación de una trayectoria suavizada mediante interpolación Catmull-Rom.

Cierre del circuito

Para evitar un salto al terminar una vuelta, el primer punto y su dirección se vuelven a colocar al final de la trayectoria.

También se calcula la longitud total del circuito y se establece una velocidad constante de:

12 unidades por segundo

La nave se mantiene aproximadamente dos unidades por encima del centro de la pista para evitar que el casco atraviese su superficie.

Movimiento automático de la nave

Durante cada ciclo se actualiza la distancia recorrida:

distanciaNaveCircuito = std::fmod(distanciaNaveCircuito + velocidadNaveCircuito * deltaTimeRover,mlongitudCircuitoNave);

El uso de fmod hace que, al terminar la longitud total del circuito, el recorrido vuelva automáticamente al inicio.

Posteriormente se determina en qué tramo se encuentra la nave y se interpola su posición entre los dos puntos correspondientes.

```
696         distanciaNaveCircuito = std::fmod(distanciaNaveCircuito
697             + velocidadNaveCircuito * deltaTimeRover, longitudCircuitoNave);
698         size_t tramoNave = 0;
699         while (tramoNave+1 < distanciasNave.size()-1
700             && distanciasNave[tramoNave+1] <= distanciaNaveCircuito)
701             ++tramoNave;
702         const float avanceTramoNave = static_cast<float>(
703             (distanciaNaveCircuito-distanciasNave[tramoNave]) /
704             (distanciasNave[tramoNave+1]-distanciasNave[tramoNave]));
705         const glm::vec3 posicionNaveCircuito = glm::mix(
706             recorridoNave[tramoNave], recorridoNave[tramoNave+1], avanceTramoNave)
707             + glm::vec3(0.0f, alturaSobrePistaNave, 0.0f);
708         const glm::vec3 frenteNaveCircuito = glm::normalize(glm::mix(
709             direccionesNave[tramoNave], direccionesNave[tramoNave+1], avanceTramoNave));
710         // El eje local +Z es la nariz. La tangente 3D incluye la pendiente.
711         // Esta base evita saltos de angulo al completar la vuelta y mantiene las alas niveladas.
712         const glm::vec3 derechaNaveCircuito = glm::normalize(glm::cross(
713             glm::vec3(0.0f,1.0f,0.0f), frenteNaveCircuito));
714         const glm::vec3 arribaNaveCircuito = glm::normalize(glm::cross(
715             frenteNaveCircuito, derechaNaveCircuito));
716         glm::mat4 orientacionNaveCircuito(1.0f);
717         orientacionNaveCircuito[0] = glm::vec4(derechaNaveCircuito,0.0f);
718         orientacionNaveCircuito[1] = glm::vec4(arribaNaveCircuito,0.0f);
719         orientacionNaveCircuito[2] = glm::vec4(frenteNaveCircuito,0.0f);
720         matrizHippocrates = glm::translate(glm::mat4(1.0f), posicionNaveCircuito)
721             * orientacionNaveCircuito;
722         matrizHippocrates = glm::scale(matrizHippocrates, glm::vec3(0.65f));
723         // Los motores heredan la traslacion y la inclinacion de la nave en cada cuadro.
724         matricesPlasma[0] = glm::translate(matrizHippocrates, glm::vec3(-1.8f,0.0f,-5.06f));
725         matricesPlasma[1] = glm::translate(matrizHippocrates, glm::vec3( 1.8f,0.0f,-5.06f));
```

Figura 4. Cálculo de la posición de la nave a partir de la distancia recorrida sobre el circuito.

Orientación de la nave

Además de cambiar su posición, la nave debe orientarse de acuerdo con la dirección de la pista.

Para ello se utiliza el vector frenteNaveCircuito, obtenido a partir de la dirección de la trayectoria. Después se calculan los vectores lateral y superior mediante productos cruzados.

Con estos tres vectores se construye una matriz de orientación:

- orientacionNaveCircuito[0] = glm::vec4(derechaNaveCircuito, 0.0f);
- orientacionNaveCircuito[1] = glm::vec4(arribaNaveCircuito, 0.0f);
- orientacionNaveCircuito[2] = glm::vec4(frenteNaveCircuito, 0.0f);

Finalmente, esta orientación se combina con la posición de la nave.

Este procedimiento permite que la nariz de la nave siga la dirección del circuito y, debido a que el vector de frente también contiene la variación de altura de la pista, la nave se inclina correctamente al subir y bajar la rampa.

Motores de la nave

Las matrices utilizadas para representar los dos motores también se actualizan a partir de matrizHippocrates.

Por esta razón, los motores heredan tanto la traslación como la orientación de la nave durante las curvas y las rampas.

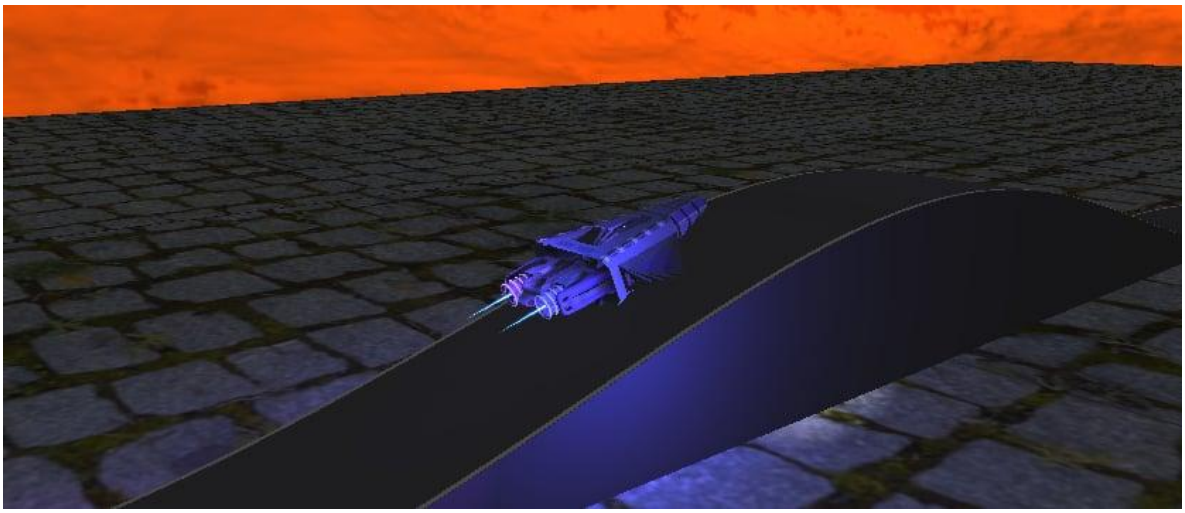


Figura 5. Nave espacial desplazándose automáticamente por uno de los tramos rectos del circuito.

Ejercicio 2. Animación automática del satélite

Para el segundo ejercicio se utilizó un satélite formado por cinco modelos independientes:

- Cuerpo del satélite
- Ala derecha 1
- Ala derecha 2
- Ala izquierda 1
- Ala izquierda 2

Estos modelos permiten conservar la estructura jerárquica trabajada anteriormente y controlar independientemente las diferentes articulaciones de las alas.

Los cinco modelos son cargados desde archivos FBX antes de iniciar la animación.

Trayectoria del satélite

Para automatizar su vuelo se utilizaron dos fases independientes.

faseRecorridoSatelite controla el desplazamiento del satélite y faseAlasSatelite controla el movimiento de sus alas.

El recorrido completo tarda aproximadamente 30 segundos, mientras que el ciclo completo de apertura y cierre de las alas tarda aproximadamente 12 segundos.

Movimiento circular

La posición del satélite se calcula mediante seno y coseno, por lo tanto, el satélite realiza una trayectoria circular en el plano XY manteniendo constante su coordenada Z. El radio utilizado es de 14 unidades.

```
767         faseRecorridoSatelite = std::fmod(faseRecorridoSatelite
768             + (vueltaSatelite / 30.0) * deltaTimeRover, vueltaSatelite);
769         faseAlasSatelite = std::fmod(faseAlasSatelite
770             + (vueltaSatelite / 12.0) * deltaTimeRover, vueltaSatelite);
771         // Inicio permite volver al comienzo de la animacion automatica.
772         if (mainWindow.getsKeys()[GLFW_KEY_HOME])
773         {
774             faseRecorridoSatelite = 0.0;
775             faseAlasSatelite = 0.0;
776         }
777         posicionSatelite = glm::vec3(
778             18.0f + radioRecorridoSatelite * static_cast<float>(std::cos(faseRecorridoSatelite)),
779             22.0f + radioRecorridoSatelite * static_cast<float>(std::sin(faseRecorridoSatelite)),
780             45.0f);
```

Figura 6. Cálculo automático de la posición del satélite mediante una trayectoria circular.

Automatización del movimiento de las alas

Anteriormente, las articulaciones de las alas podían modificarse mediante teclado. En esta versión sus ángulos se actualizan automáticamente.

Para producir un movimiento gradual se utiliza:

const GLfloat plegadoSatelite = 0.5f * (1.0f - cos(faseAlasSatelite));

Este valor cambia suavemente entre 0 y 1 y después vuelve a 0, por lo que las alas no cambian bruscamente de posición.

```
783         const GLfloat plegadoSatelite = 0.5f * (1.0f - static_cast<float>(std::cos(faseAlasSatelite)));
784         giroAlaDer1 = giroAlaIzq1 = 20.0f * plegadoSatelite;
785         giroAlaDer2 = giroAlaIzq2 = 30.0f * plegadoSatelite;
```

Figura 7. Cálculo automático de los ángulos utilizados para abrir y cerrar las alas del satélite.

Jerarquía del satélite

La posición completa del satélite parte de una matriz padre. El cuerpo utiliza directamente esta transformación, mientras que cada ala se coloca utilizando su propio pivote.

Las segundas alas dependen de las primeras. Por ejemplo, AlaDer2 utiliza como matriz base alaPadreDer1. De esta forma la segunda sección hereda automáticamente el movimiento aplicado a la primera.

Los lados izquierdo y derecho utilizan signos opuestos en sus rotaciones, permitiendo un plegado simétrico.

```
961         color = glm::vec3(0.08f, 0.25f, 0.55f);
962         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
963
964         glm::mat4 alaPadreDer1 = glm::translate(satelitePadre, pivoteAlaDer1);
965         alaPadreDer1 = glm::rotate(alaPadreDer1, glm::radians(giroAlaDer1), glm::vec3(0.0f, 0.0f, 1.0f));
966         model = glm::scale(alaPadreDer1, glm::vec3(escalaSateliteFBX));
967         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
968         AlaDer1_M.RenderModel();
969
970         glm::mat4 alaPadreDer2 = glm::translate(alaPadreDer1, pivoteAlaDer2 - pivoteAlaDer1);
971         alaPadreDer2 = glm::rotate(alaPadreDer2, glm::radians(giroAlaDer2), glm::vec3(0.0f, 0.0f, 1.0f));
972         model = glm::scale(alaPadreDer2, glm::vec3(escalaSateliteFBX));
973         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
974         AlaDer2_M.RenderModel();
975
976         glm::mat4 alaPadreIzq1 = glm::translate(satelitePadre, pivoteAlaIzq1);
977         alaPadreIzq1 = glm::rotate(alaPadreIzq1, glm::radians(-giroAlaIzq1), glm::vec3(0.0f, 0.0f, 1.0f));
978         model = glm::scale(alaPadreIzq1, glm::vec3(escalaSateliteFBX));
979         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
980         AlaIzq1_M.RenderModel();
981
982         glm::mat4 alaPadreIzq2 = glm::translate(alaPadreIzq1, pivoteAlaIzq2 - pivoteAlaIzq1);
983         alaPadreIzq2 = glm::rotate(alaPadreIzq2, glm::radians(-giroAlaIzq2), glm::vec3(0.0f, 0.0f, 1.0f));
984         model = glm::scale(alaPadreIzq2, glm::vec3(escalaSateliteFBX));
985         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
986         AlaIzq2_M.RenderModel();
987
```

Figura 8. Jerarquía utilizada para conservar la relación padre-hijo entre las diferentes secciones de las alas.

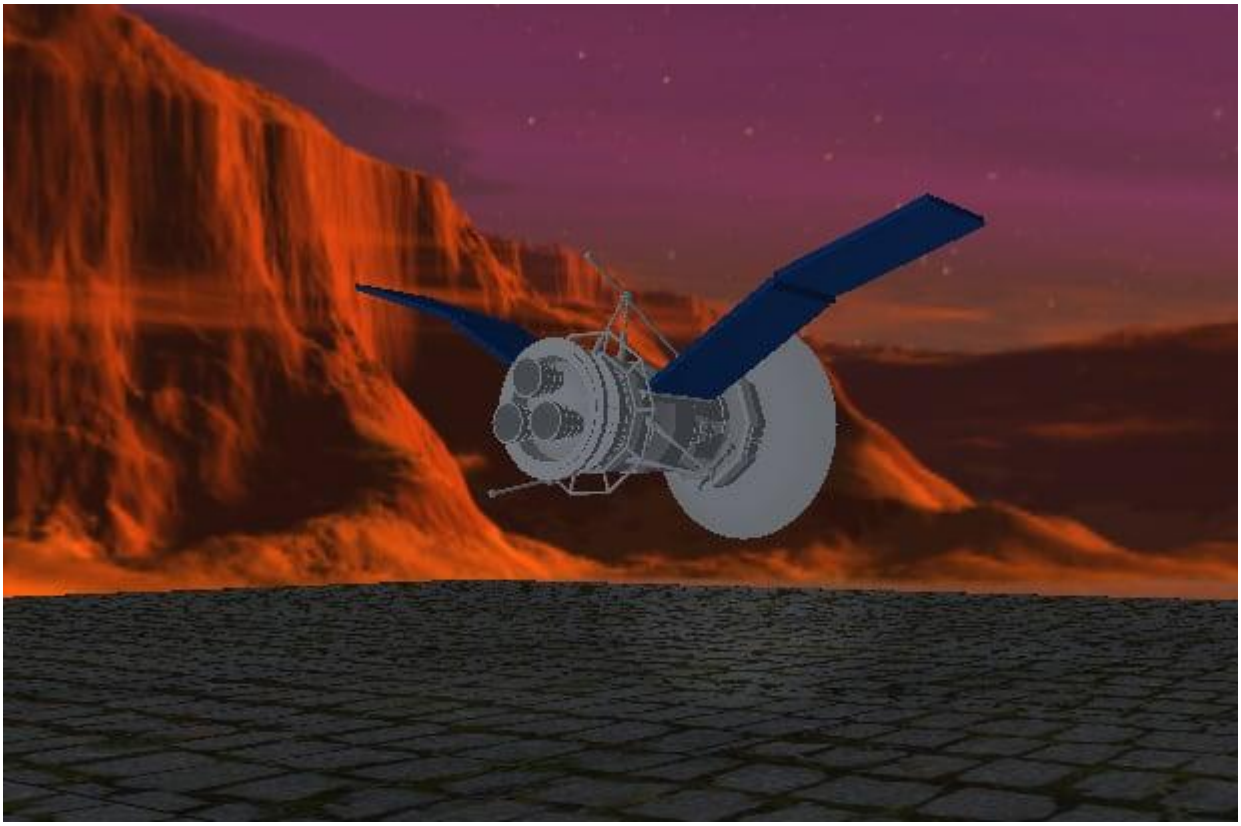


Figura 9. Satélite desplazándose por su trayectoria con las alas en una posición abierta.

Resultados

Una de las principales dificultades del primer ejercicio fue conseguir que la nave siguiera diferentes tipos de segmentos sin presentar cambios bruscos de

dirección. Utilizar directamente los puntos originales del circuito podía producir transiciones menos suaves, por lo que se generó una trayectoria con interpolación Catmull-Rom.

También fue necesario considerar la altura de la pista. La nave no solamente debe modificar sus coordenadas X y Z, sino también su posición en Y y su orientación al llegar a la rampa. Por esta razón se utilizó una dirección tridimensional para construir la matriz de orientación.

Para el satélite, la principal dificultad fue sustituir el movimiento mediante teclado por una animación automática sin perder la jerarquía de sus alas. Se conservaron los mismos pivotes y relaciones padre-hijo, pero los ángulos ahora se calculan automáticamente utilizando una función periódica.

Problemas presentados

En el ejercicio del rover, uno de los principales problemas que tuve fue completar las seis ruedas sin perder la relación jerárquica con la base. Cada rueda debía contar con su propio soporte, mantener una posición específica y utilizar una articulación diferente para poder girar de forma independiente.

También fue necesario que manejara correctamente las matrices auxiliares para evitar que el movimiento de una rueda afectara accidentalmente a las demás.

En el ejercicio de la sonda espacial, la principal dificultad que tuve fue organizar la jerarquía para que todas las piezas heredaran la rotación general del cuerpo, pero al mismo tiempo los paneles solares y la antena conservaran movimientos independientes.

También tuve que ajustar posiciones y escalas para conseguir que las piezas permanecieran visualmente conectadas al cuerpo central.

Conclusión:

a) Complejidad y explicación: La principal complejidad fue sincronizar la posición y orientación de los modelos con trayectorias que cambian continuamente. En la nave se generó una ruta suavizada a partir del circuito y se utilizó su dirección para orientarla correctamente durante rectas, curvas y rampas. En el satélite se combinaron dos animaciones independientes: una trayectoria circular para todo el modelo y un ciclo periódico para abrir y cerrar sus alas, manteniendo la jerarquía de sus componentes.

b) Comentarios generales: Estos ejercicios permitieron trabajar con animaciones más completas que las realizadas únicamente mediante condiciones o entradas del teclado. La nave demostró cómo un modelo puede seguir una trayectoria definida por varios tipos de movimiento, mientras que el satélite permitió automatizar movimientos que anteriormente dependían directamente del usuario. También se

reforzó el uso de jerarquías para conservar correctamente el movimiento de diferentes partes de un mismo modelo.

c) Conclusión: Se concluye que una trayectoria compleja puede construirse combinando distintos segmentos y posteriormente suavizarse para obtener un movimiento más continuo. En el caso de la nave, el uso de los puntos del circuito, la interpolación y el cálculo de su dirección permitió recorrer correctamente rectas, curvas y rampas sin separar el modelo de la pista. También se concluye que una animación previamente controlada mediante teclado puede transformarse en una animación automática utilizando variables que cambien con el tiempo. En el satélite, esto permitió conservar la estructura jerárquica de sus alas mientras sus movimientos se realizan de manera continua y simétrica. Finalmente, se comprobó que es posible ejecutar varias transformaciones de manera simultánea, ya que la nave modifica su posición y orientación durante el circuito, mientras que el satélite combina el desplazamiento de todo el modelo con la rotación independiente de sus componentes.

Bibliografía

- OpenGL-Tutorial.org. (s. f.). *Tutorial 3: Matrices*. https://www.opengl-tutorial.org/es/beginners-tutorials/tutorial-3-matrices/
- Lluch Crespo, J. (2008). *Animación en OpenGL* [Video didáctico]. Universitat Politècnica de València. https://www.upv.es/visor/media/4365a92a-24f0-c84a-a5c0-b4711383db39/c
- Castillo Márquez, D. I. (2026). *Círculo unitario en movimiento* [Material didáctico interactivo]. Universidad Autónoma de Nayarit. https://daliacastillo.github.io/circulo-unitario/